

ES6 JavaScript Concepts & React Hooks

A Comprehensive, Beginner-Friendly Guide for Presentations &
Technical Mastery

Prepared for Technical Training Submission & Professional Development
Version 1.0 | Complete Reference Guide

Table of Contents

1. Basic ES6 Concepts	Section 1
1.1 let, var and const	Page 2
1.2 Arrow Functions (=>)	Page 3
1.3 Template Literals (\${ })	Page 4
1.4 Destructuring (Arrays & Objects)	Page 5
1.5 Spread Operator (...)	Page 6
1.6 Rest Parameters (...)	Page 7
1.7 Default Parameters	Page 8
1.8 Object and Array Methods	Page 9
1.9 Modules (import & export)	Page 10
1.10 Promises	Page 11
1.11 async / await	Page 12
1.12 Ternary Operator	Page 13
1.13 Optional Chaining (?.)	Page 14
2. ES6 Classes	Section 2
2.1 ES6 Classes & Constructors	Page 15
3. Important Array Methods	Section 3
3.1 map(), filter(), find(), forEach(), reduce() & Comparison	Page 16
4. Objects in JavaScript	Section 4
4.1 Creation, Nested Objects & Object Methods	Page 18
5. React Hooks	Section 5
5.1 Introduction to React Hooks & useState	Page 19
5.2 useEffect Hook	Page 20
6. Quick Revision Reference	Section 6

1. Basic ES6 Concepts

What is ES6? ECMAScript 6 (ES6), also known as ECMAScript 2015, represents a major milestone in the evolution of JavaScript. It introduced syntax enhancements and powerful programming constructs that transformed JavaScript from a simple scripting language into a robust, enterprise-grade programming language capable of building massive, scalable applications.

When was ES6 introduced? ES6 was officially approved and published in June 2015 by Ecma International.

Why ES6 is important: Modern JavaScript development relies heavily on ES6 features. It dramatically reduces boilerplate code, improves code readability, enforces cleaner scoping rules, and introduces modern paradigms like modular architecture and asynchronous programming.

Major improvements: Block-scoped declarations (`let`, `const`), arrow functions, template literals, destructuring assignment, spread/rest operators, promises, `async/await`, classes, and native modules.

1.1 let, var and const

JavaScript provides three keywords for variable declaration: `var`, `let`, and `const`. Understanding their scoping rules, hoisting behavior, and reassignment capabilities is fundamental to writing bug-free code.

FEATURE	VAR	LET	CONST
Scope	Function scoped	Block scoped	Block scoped
Hoisting	Yes (initialized as undefined)	Yes (uninitialized / Temporal Dead Zone)	Yes (uninitialized / Temporal Dead Zone)
Reassignment	Allowed	Allowed	Not allowed
Redeclaration	Allowed in same scope	Error in same scope	Error in same scope

Syntax

```
let variableName = value;
const constantName = value;
var variableName = value;
```

Example

```
let age = 21;
age = 22; // Valid reassignment

const name = "Pavani";
// name = "Rahul"; // TypeError: Assignment to constant variable

var city = "Bangalore";
city = "Hyderabad"; // Valid reassignment

console.log(age);
console.log(name);
console.log(city);
```

OUTPUT:

22 Pavani Hyderabad

Explanation of Output: `age` was declared with `let` and successfully reassigned from 21 to 22. `name` was declared with `const` and retained its initial value "Pavani". `city` was declared with `var` and updated to "Hyderabad". All values printed successfully to the console.

KEY TAKEAWAY

"Always use `const` by default for values that won't change, `let` for variables that need to be reassigned, and avoid `var` entirely in modern JavaScript to prevent scope leakage bugs."

1.2 Arrow Functions (=>)

Arrow functions provide a concise syntax for writing function expressions in JavaScript. They do not bind their own `this` context, making them ideal for callbacks and array methods.

- **Shorter syntax:** Eliminates the function keyword.
- **Implicit return:** Single-expression functions can omit curly braces and the `return` keyword.
- **Lexical this:** Inherits `this` from the surrounding execution context.

Syntax

```
const functionName = (parameters) => {  
  // code block  
};  
  
// Shorter implicit return syntax  
const shortName = (parameters) => expression;
```

Example

```
const add = (a, b) => {  
  return a + b;  
};  
  
console.log(add(10, 20));  
  
// Shorter form demonstration  
const multiply = (a, b) => a * b;  
console.log(multiply(5, 4));
```

OUTPUT:

30 20

Explanation: The first function `add` takes parameters `a` and `b`, executes the block, and returns 30. The second function `multiply` uses implicit return to calculate `5 * 4` and outputs 20 instantly.

KEY TAKEAWAY

"Arrow functions provide a clean, concise syntax and lexical scoping for this, making callback functions much easier to read and maintain."

1.3 Template Literals (\${ })

Template literals are string literals enclosed in backticks (``) instead of single or double quotes. They allow embedded expressions, variable interpolation, and multi-line strings without concatenation operators.

Syntax

```
`Hello ${variableName}`
```

Example

```
const name = "Pavani";  
const age = 21;  
  
console.log(`My name is ${name} and I am ${age} years old.`);  
  
const multiLine = `This is line one  
This is line two`;  
console.log(multiLine);
```

OUTPUT:

```
My name is Pavani and I am 21 years old. This is line one This is line two
```

Explanation: Backticks enable seamless interpolation of variables `${name}` and `${age}` directly inside the string, while preserving line breaks natively.

KEY TAKEAWAY

"Template literals eliminate messy string concatenation using plus operators by allowing clean variable interpolation and multi-line strings with backticks."

1.4 Destructuring

Destructuring is an expression syntax that unpacks values from arrays or properties from objects into distinct variables, dramatically simplifying data extraction.

Syntax

```
// Array Destructuring
const [item1, item2] = array;

// Object Destructuring
const { prop1, prop2 } = object;
```

Array Example

```
const colors = ["Red", "Blue", "Green"];

const [first, second, third] = colors;

console.log(first);
console.log(second);
console.log(third);
```

OUTPUT:

Red Blue Green

Object Example

```
const student = {
  name: "Pavani",
  age: 21
};

const { name, age } = student;

console.log(name);
console.log(age);
```

OUTPUT:

Pavani 21

Explanation: Array destructuring assigns array items positionally to variables, whereas object destructuring matches property names directly to variable identifiers.

KEY TAKEAWAY

"Destructuring allows us to unpack data from arrays and objects cleanly into distinct variables with minimal code."

1.5 Spread Operator (...)

The spread operator (...) unpacks elements of an iterable (like an array) or properties of an object. It is widely used for copying arrays, combining arrays, and merging object properties.

Syntax

```
const combinedArray = [...array1, ...array2];
const copiedObject = { ...originalObject, newProp: value };
```

Example

```
const numbers1 = [1, 2, 3];
const numbers2 = [4, 5, 6];

const numbers = [...numbers1, ...numbers2];
console.log(numbers);

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };
console.log(obj2);
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6] { a: 1, b: 2, c: 3 }
```

Explanation: The spread operator successfully expanded numbers1 and numbers2 into a single array numbers, and merged obj1 properties into obj2.

KEY TAKEAWAY

"The spread operator provides a clean, immutable way to copy and merge arrays and objects without mutating original data."

1.6 Rest Parameters (...)

Rest parameters allow a function to accept an indefinite number of arguments as an array, providing a cleaner alternative to the old arguments object.

Syntax

```
function functionName(...parameters) {  
    // code  
}
```

Example

```
function sum(...numbers) {  
    return numbers.reduce((total, num) => total + num, 0);  
}  
  
console.log(sum(10, 20, 30));  
console.log(sum(5, 15, 25, 35));
```

OUTPUT:

60 80

Explanation: The rest parameter `...numbers` collects all passed arguments into an array, allowing `reduce()` to calculate the total sum regardless of how many arguments are provided.

Spread vs Rest: The *Spread* operator unpacks/expands elements (e.g. passing array items to a function), whereas the *Rest* parameter packs multiple arguments into an array.

KEY TAKEAWAY

"Rest parameters collect multiple function arguments into a neat array, enabling flexible functions that accept variable numbers of inputs."

1.7 Default Parameters

Default parameters allow function parameters to be initialized with default values if no argument or undefined is passed.

Syntax

```
function functionName(parameter = defaultValue) {  
    // code  
}
```

Example

```
function greet(name = "Guest") {  
    return `Hello ${name}`;  
}  
  
console.log(greet());  
console.log(greet("Pavani"));
```

OUTPUT:

Hello Guest Hello Pavani

Explanation: When `greet()` is called without arguments, it defaults to "Guest". When called with "Pavani", it uses the provided argument.

KEY TAKEAWAY

"Default parameters prevent unexpected undefined bugs by assigning fallback values cleanly right in the function signature."

1.8 Object and Array Methods

JavaScript provides robust built-in methods for manipulating arrays and inspecting objects efficiently.

METHOD	PURPOSE
<code>push()</code>	Adds element to end
<code>pop()</code>	Removes last element
<code>shift()</code>	Removes first element
<code>unshift()</code>	Adds element to beginning
<code>slice()</code>	Returns part of an array
<code>includes()</code>	Checks whether value exists
<code>Object.keys()</code>	Returns object keys
<code>Object.values()</code>	Returns object values
<code>Object.entries()</code>	Returns key-value pairs

Example

```
const fruits = ["Apple", "Banana"];
fruits.push("Mango");
console.log(fruits);

const user = { name: "Pavani", age: 21 };
console.log(Object.keys(user));
console.log(Object.values(user));
```

OUTPUT:

```
[ 'Apple', 'Banana', 'Mango' ] [ 'name', 'age' ] [ 'Pavani', 21 ]
```

KEY TAKEAWAY

"Built-in array and object inspection methods make data manipulation and property auditing fast and declarative."

1.9 Modules

ES6 modules allow developers to break large programs into separate, reusable files using `export` and `import` statements.

- **Named Export:** Export multiple values using explicit names.
- **Default Export:** Export a single primary value per module.

Example

```
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;

// app.js
import { add, subtract } from "./math.js";
console.log(add(10, 20));
console.log(subtract(30, 10));
```

OUTPUT:

30 20

Explanation: Modules help organize large applications by enforcing separation of concerns, keeping code clean, testable, and modular.

KEY TAKEAWAY

"ES6 modules allow us to organize code across multiple files by exporting and importing functions and variables cleanly."

1.10 Promises

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. A promise exists in one of three states: *Pending*, *Fulfilled*, or *Rejected*.

Syntax

```
const promise = new Promise((resolve, reject) => {  
  // asynchronous operation  
});
```

Example

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve("Operation successful");  
  } else {  
    reject("Operation failed");  
  }  
});  
  
promise  
  .then(result => console.log(result))  
  .catch(error => console.log(error));
```

OUTPUT:

Operation successful

Explanation: The promise executes asynchronously. Since success is true, it calls `resolve`, triggering the `.then()` callback handler.

KEY TAKEAWAY

"Promises handle asynchronous operations cleanly, replacing nested callback hell with structured `.then()` and `.catch()` chains."

1.11 async/await

The `async/await` syntax is syntactic sugar built on top of Promises. It allows asynchronous code to be written in a synchronous, highly readable linear style.

Example

```
const getData = async () => {
  try {
    const response = await fetch("https://dummyjson.com/products/1");
    const data = await response.json();
    console.log("Fetched product:", data.title);
  } catch (error) {
    console.log("Error:", error);
  }
};

getData();
```

OUTPUT:

Fetched product: iPhone 9

Flow Step-by-Step: 1. `async` declares an asynchronous function returning a Promise. 2. `await` pauses function execution until the `fetch` promise resolves. 3. Response is parsed into JSON. 4. Product title is logged to the console.

KEY TAKEAWAY

"async/await makes asynchronous code read like synchronous code, eliminating promise chain nesting and improving error handling."

1.12 Ternary Operator

The ternary operator is a concise shorthand for simple `if...else` statements.

Syntax

```
condition ? valueIfTrue : valueIfFalse;
```

Example

```
const age = 20;  
const result = age >= 18 ? "Adult" : "Minor";  
console.log(result);
```

OUTPUT:

Adult

Explanation: Since `age >= 18` evaluates to true, the ternary operator assigns "Adult" to `result`.

KEY TAKEAWAY

"The ternary operator provides a clean, inline alternative for simple conditional assignments."

1.13 Optional Chaining (?.)

Optional chaining (?.) safely extracts nested property values without throwing a `TypeError` if an intermediate reference is `null` or `undefined`.

Example

```
const user = {  
  name: "Pavani",  
  address: {  
    city: "Bangalore"  
  }  
};  
  
console.log(user.address?.city);  
console.log(user.contact?.phone);
```

OUTPUT:

Bangalore undefined

Explanation: `user.address?.city` successfully finds "Bangalore". `user.contact?.phone` safely evaluates to `undefined` instead of crashing with a runtime error because `contact` does not exist.

KEY TAKEAWAY

"Optional chaining prevents runtime crashes when accessing deeply nested object properties that may be undefined."

2. ES6 Classes

2.1 ES6 Classes & Constructors

ES6 introduced class syntax to provide a clean, object-oriented model in JavaScript, acting as syntactic sugar over prototype-based inheritance.

- **Constructor:** Special method for initializing object properties.
- **Methods:** Functions defined inside a class to perform operations.
- **new keyword:** Used to instantiate object instances from a class.

Syntax

```
class ClassName {  
  constructor(parameters) {  
    this.property = parameters;  
  }  
  methodName() {  
    // code  
  }  
}
```

Example

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    return `Hello, I am ${this.name}`;  
  }  
}  
  
const student = new Student("Pavani", 21);  
console.log(student.greet());
```

OUTPUT:

Hello, I am Pavani

Explanation: The Student class defines a constructor and a greet() method. Instantiating with new Student("Pavani", 21) creates an object instance that successfully invokes greet().

KEY TAKEAWAY

"ES6 classes provide a clean, familiar object-oriented syntax for creating objects and managing inheritance in JavaScript."

3. Important Array Methods

Modern JavaScript provides powerful functional array iteration methods that make data processing declarative and concise.

3.1 map()

Purpose: Transforms each element in an array and returns a **brand new array** of the same length.

Syntax

```
const newArray = array.map((element) => transformedElement);
```

Example

```
const numbers = [1, 2, 3, 4];  
const doubled = numbers.map(num => num * 2);  
console.log(doubled);
```

OUTPUT:

```
[2, 4, 6, 8]
```

3.2 filter()

Purpose: Evaluates a condition for each element and returns a new array containing only elements that pass the test.

Syntax

```
const filteredArray = array.filter((element) => condition);
```

Example

```
const numbers = [1, 2, 3, 4, 5];  
const evenNumbers = numbers.filter(num => num % 2 === 0);  
console.log(evenNumbers);
```

OUTPUT:

```
[2, 4]
```

3.3 find()

Purpose: Returns the **first matching element** in the array that satisfies the provided testing function.

Syntax

```
const foundItem = array.find((element) => condition);
```

Example

```
const numbers = [10, 20, 30, 40];  
const result = numbers.find(num => num > 20);  
console.log(result);
```

OUTPUT:

```
30
```

3.4 forEach()

Purpose: Executes a provided function once for each array element. Does **not** create a new array (returns undefined).

Syntax

```
array.forEach((element) => { /* code */ });
```

Example

```
const fruits = ["Apple", "Banana", "Mango"];
fruits.forEach(fruit => console.log(fruit));
```

OUTPUT:

Apple Banana Mango

3.5 reduce()

Purpose: Applies a function against an accumulator and each element in the array to reduce it to a **single output value**.

Syntax

```
const result = array.reduce((accumulator, currentValue) => {
  return accumulator + currentValue;
}, initialValue);
```

Example

```
const numbers = [10, 20, 30];
const total = numbers.reduce((sum, num) => sum + num, 0);
console.log(total);
```

OUTPUT:

60

3.6 Array Methods Summary & Comparison

METHOD	MAIN PURPOSE	RETURNS	MEMORY TRICK
<code>map()</code>	Transform elements	New Array	Transform
<code>filter()</code>	Select elements	New Array	Select
<code>find()</code>	Find first match	Single Element	First match
<code>forEach()</code>	Execute code for each	undefined	Perform action
<code>reduce()</code>	Combine values	Single Result	Combine

KEY TAKEAWAY

"Choose the right array method based on your goal: `map` to transform, `filter` to select, `find` for the first match, `forEach` for side effects, and `reduce` to aggregate values."

4. Objects in JavaScript

4.1 Creation, Nested Objects & Object Methods

Objects store keyed collections of data and complex entities. Properties can be accessed using dot notation or bracket notation.

Example

```
const student = {
  name: "Pavani",
  age: 21,
  address: {
    city: "Bangalore",
    state: "Karnataka"
  },
  greet() {
    return `Hello, my name is ${this.name}`;
  }
};

console.log(student.name);
console.log(student["age"]);
console.log(student.address.city);
console.log(student.greet());
```

OUTPUT:

Pavani 21 Bangalore Hello, my name is Pavani

Explanation: `this` keyword refers to the current object context, allowing methods to access properties of the same object instance.

KEY TAKEAWAY

"Objects encapsulate related data and behavior into properties and methods, supporting nesting and context-aware execution with `this`."

5. React Hooks

What are React Hooks? Introduced in React 16.8, Hooks allow functional components to use state, lifecycle effects, context, and other React features without writing class components.

- Hooks must always be called at the top level of React components.
- Common hooks include `useState`, `useEffect`, `useContext`, `useRef`, `useMemo`, and `useCallback`.

5.1 useState Hook

useState is a React Hook that lets you add state variables to functional components. It returns an array with two values: the current state and a function to update it.

Syntax

```
const [state, setState] = useState(initialValue);
```

Complete Example

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  const increaseCount = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <span>Count: {count}</span>
      <button>Increase</button>
    </div>
  );
}

export default Counter;
```

OUTPUT:

Rendering UI with initial Count: 0 [User clicks 'Increase' button] State updates to 1 -> Component re-renders -> Displays Count: 1

Explanation: When setCount is invoked, React updates the state variable count and triggers a re-render of the Counter component to reflect the new value on the UI.

KEY TAKEAWAY

"useState brings local component state to functional components, automatically triggering re-renders whenever state updates."

5.2 useEffect Hook

`useEffect` lets you perform side effects in functional components, such as data fetching, subscriptions, or manual DOM manipulations.

Syntax

```
useEffect(() => {  
    // side effect code  
}, [dependencies]);
```

Complete Example

```
import React, { useEffect, useState } from "react";

function Products() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetch("https://dummyjson.com/products")
      .then(response => response.json())
      .then(data => setProducts(data.products));
  }, []);

  return (
```

Products

```
    {products.slice(0, 3).map(product => (
      <div>
        {product.title}
      </div>
    ))}

  );
}

export default Products;
```

OUTPUT:

Component mounts -> useEffect runs -> API request sent -> Data received -> setProducts updates state -> Component re-renders with products.

Explanation Flow: 1. Component renders for the first time. 2. useEffect executes after render (empty dependency array [] ensures it runs only once on mount). 3. API fetches product data. 4. setProducts() updates state, triggering a re-render to display products.

KEY TAKEAWAY

"useEffect manages component side effects and data fetching with precise control governed by dependency arrays."

6. Quick Revision Reference

ES6 Quick Revision

CONCEPT	KEY POINT
let	Block scoped, can be reassigned
const	Block scoped, cannot be reassigned
Arrow function	Short function syntax, lexical this
Template literal	String interpolation with backticks
Destructuring	Extract values from arrays/objects
Spread	Expand / copy values (...)
Rest	Collect multiple arguments into array (...)
Default parameters	Default argument fallback values
Modules	Organize and reuse code (import/export)
Promises	Handle asynchronous operations (.then/.catch)
async/await	Cleaner asynchronous code flow
Ternary	Short shorthand if/else condition
Optional chaining	Safe property access (?.)
Classes	Object-oriented syntax sugar

Array Methods Quick Revision

METHOD	REMEMBER
map()	Transform elements into new array
filter()	Select elements matching condition
find()	Find first matching element
forEach()	Perform action for each item
reduce()	Combine values into single result

React Hooks Quick Revision

HOOK	PURPOSE
useState	Manage local component state
useEffect	Handle side effects and API calls
useContext	Access React context globally
useRef	Store mutable references without re-renders
useMemo	Memoize expensive computed values
useCallback	Memoize callback functions